

1. Introduction

The Notification Management System is a software system designed to provide a flexible and scalable way to send notifications through different communication channels. The system will be developed using Object-Oriented Programming (OOP) concepts and design principles such as SOLID, along with design patterns such as Strategy, Factory, and Decorator. It will also support future expansion into a SaaS-based notification service.

1.1 Purpose

The purpose of this software is to provide a centralized notification system that allows applications or users to send notifications through different channels, such as email and other supported notification services.

The system is designed to make notification delivery easier to manage, maintain, and extend. New notification channels can be added without significantly changing the existing system.

1.2 Intended Audience

This SRS document is intended for:

- Developers – to understand the system requirements and implement the software.
- Testers – to understand the expected system behavior and prepare test cases.
- Project Manager – to plan and monitor the development process.
- Clients/Stakeholders – to understand the features, scope, and objectives of the system.
- System Designers – to understand the architecture and design requirements.

1.3 Intended Use

This SRS will be used as a guideline throughout the development lifecycle.

- Developers will use the requirements to design and implement the system.
- Testers will use the requirements to verify whether the system works correctly.
- Project managers will use the document for planning and tracking development progress.
- Stakeholders will use it to understand the expected features and limitations of the software.
- The development team will use it as a reference for future maintenance and enhancement.

1.4 Product Scope

The Notification Management System will provide the following major functionalities:

- Send notifications through different communication channels.
- Support multiple notification channels using a common interface.
- Allow new notification channels to be added easily.
- Manage notification delivery through a structured architecture.
- Store relevant notification and user information in a database.
- Provide authentication and authorization.
- Support asynchronous notification processing using a queue and worker.
- Support multi-tenant functionality using tenant identification.
- Apply rate limiting to control excessive requests.
- Provide monitoring and system status information.
- Support future subscription and billing features.

The main goal of the system is to provide a reliable, maintainable, scalable, and extensible notification platform.

1.5 Risk Definitions

The major risks associated with the Notification Management System include:

- **Server Failure:** If the server becomes unavailable, notifications may not be processed.
- **Database Failure:** Database problems may cause loss or unavailability of notification-related data.
- **Notification Delivery Failure:** Email or other external notification services may fail to deliver messages.
- **Security Breach:** Unauthorized access may expose user or tenant information.
- **High Traffic:** A large number of requests may overload the system if rate limiting and queue management are not properly implemented.
- **Third-Party Service Failure:** External services used for notification delivery may become temporarily unavailable.
- **Data Loss:** Unexpected system or database failures may result in loss of important data.
- **Scalability Issues:** Increasing numbers of users or notifications may affect system performance if the architecture is not properly designed.